

Translation Schemes

- In a syntax-directed definition, we do not say anything about the evaluation times of the semantic rules (when the semantic rules associated with a production should be evaluated?).
- A translation scheme is a context-free grammar in which:
 - attributes are associated with the grammar symbols and
 - semantic actions enclosed between braces $\{\}$ are inserted within the right sides of productions.

• *Ex:* $A \rightarrow \{ \dots \} X \{ \dots \} Y \{ \dots \}$



Semantic Actions

Translation Schemes (cont.)

- ◆ When designing a translation scheme, some restrictions should be observed **to ensure that an attribute value is available when a semantic action refers to that attribute.**
- ◆ These restrictions (motivated by L-attributed definitions) ensure that a semantic action does not refer to an attribute that has not yet computed.
- ◆ In translation schemes, we use *semantic action* terminology instead of *semantic rule* terminology used in syntax-directed definitions.
- ◆ The position of the semantic action on the right side indicates **when** that semantic action will be evaluated.

Translation Schemes (cont.)

- A CFG with “**semantic actions**” *embedded* into its productions.
Useful for binding the order of evaluation into the parse-tree.
- As before semantic actions might refer to the attributes of the symbols.

Example.

$\text{expr} \rightarrow \text{expr} + \text{term} \{ \text{print}(\text{"+"}) \}$

$\text{expr} \rightarrow \text{expr} - \text{term} \{ \text{print}(\text{"-"}) \}$

$\text{expr} \rightarrow \text{term}$

$\text{term} \rightarrow 0 \{ \text{print}(\text{"0"}) \}$

...

$\text{term} \rightarrow 9 \{ \text{print}(\text{"9"}) \}$

=> Traversing a parse tree for a Translation-Scheme produces the translation. (we will employ only DFS)

=> Translation schemes also help in materializing L-attributed Definitions.

Translation Schemes - example



- The production and semantic rule:

Production

$T \rightarrow T1 * F$

semantic rule

$T.val := T1.val * F.val$



Yield the following production and semantic action:

$T \rightarrow T1 * F$

$\{T.val := T1.val * F.val\}$

Writing Translation Schemes



- ◆ Start with a Syntax-Directed Definition.
- ◆ *In General*: Make sure that we never refer to an attribute that has not been defined already.
- ◆ For S-Attributed Definitions, we simply put all semantic rules into {...} at the rightmost of each production.
- ◆ If both inherited and synthesized attributes are involved:
 - ◆ An inherited attribute for a symbol on the RHS of a production must be computed in an action before that symbol.
 - ◆ An action must not refer to a synthesized attribute of a symbol that is to the right.
 - ◆ A synthesized attribute for the NT on the LHS can only be computed after all attributes it references are already computed. (the action for such attributes is typically placed in the rightmost end of the production).
- ◆ L-attributed definitions are suited for the above...

Examples


$$S \rightarrow A_1 \{S.s = A_1.s + A_2.s\} A_2$$
$$A \rightarrow \textcolor{brown}{a} \{A.s = 1\}$$

This will not work...

On the other hand this is good:

$$S \rightarrow A_1 A_2 \{S.s = A_1.s + A_2.s\}$$
$$A \rightarrow \textcolor{brown}{a} \{A.s = 1\}$$

Examples II


$$S \rightarrow A_1 A_2 \{A_1.in = 1; A_2.in = 2\}$$
$$A \rightarrow a \{print(A.in)\}$$

This will not work...

On the other hand this is good:

$$S \rightarrow \{A_1.in = 1; A_2.in = 2\} A_1 A_2$$
$$A \rightarrow a \{print(A.in)\}$$

Or even:

$$S \rightarrow \{A_1.in = 1\} A_1 \{A_2.in = 2\} A_2$$
$$A \rightarrow a \{print(A.in)\}$$

Translation Schemes for S-attributed Definitions

- If our syntax-directed definition is S-attributed, the construction of the corresponding translation scheme will be simple.
- Each associated semantic rule in a S-attributed syntax-directed definition will be inserted as a semantic action into the end of the right side of the associated production.

Production

$E \rightarrow E_1 + T$

Semantic Rule

$E.val = E_1.val + T.val$

➔ a production of

a syntax directed definition

⇓

$E \rightarrow E_1 + T \{ E.val = E_1.val + T.val \}$

➔ the production of the corresponding

translation scheme

Inherited Attributes in Translation Schemes



- ◆ If a translation scheme has to contain both synthesized and inherited attributes, we have to observe the following rules:
 - ◆ An inherited attribute of a symbol on the right side of a production must be computed in a semantic action before that symbol.
 - ◆ A semantic action must not refer to a synthesized attribute of a symbol to the right of that semantic action.
 - ◆ A synthesized attribute for the non-terminal on the left can only be computed after all attributes it references have been computed (we normally put this semantic action at the end of the right side of the production).
- ◆ With a L-attributed syntax-directed definition, it is always possible to construct a corresponding translation scheme which satisfies these three conditions (This may not be possible for a general syntax-directed translation).

An example



- ◆ The following translation scheme does not satisfy these three requirements:

$$\begin{aligned} S &\rightarrow A1 \ A2 \ \{A1.in = 1; A2.in = 2\} \\ A &\rightarrow a \ \quad \quad \{print(A.in)\} \end{aligned}$$

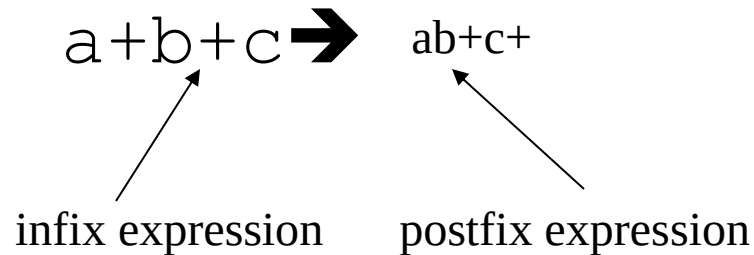
A Translation Scheme Example



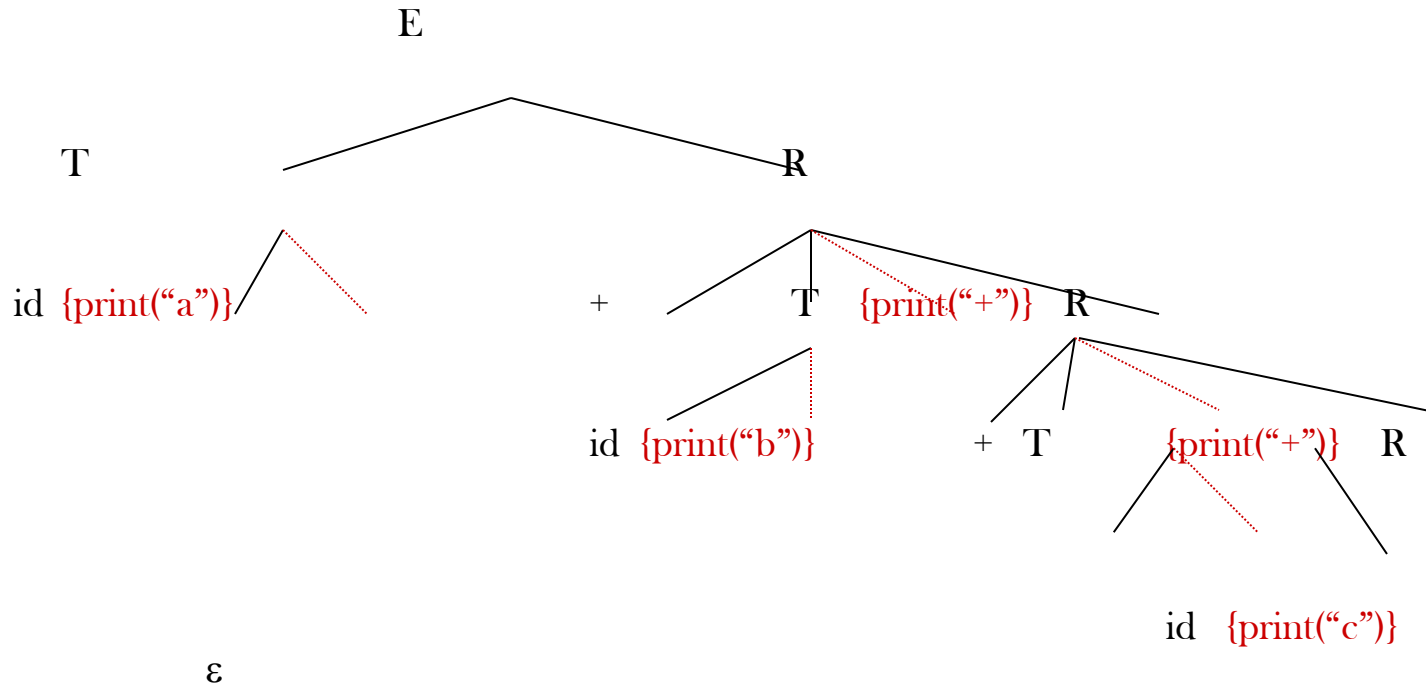
A simple translation scheme that converts infix expressions to the corresponding postfix expressions.

$$E \rightarrow T R$$
$$R \rightarrow + T \{ \text{print}(\text{"+"}) \} R1$$
$$R \rightarrow \varepsilon$$
$$T \rightarrow \text{id} \{ \text{print}(\text{id.name}) \}$$

What kind of expression?



A Translation Scheme Example (cont.)



The depth first traversal of the parse tree (executing the semantic actions in that order) will produce the postfix representation of the infix expression.



Derivation with semantic actions

Left-most derivation:

- $E \rightarrow T R$
- $\rightarrow \text{id} \{ \text{print}(\text{id.name}) \} R$
- $\rightarrow \text{id} \{ \text{print}(\text{id.name}) \} + T \{ \text{print}("+") \} R$
- $\rightarrow \text{id} \{ \text{print}(\text{id.name}) \} + \text{id} \{ \text{print}(\text{id.name}) \} \{ \text{print}("+") \} R$
- $\rightarrow \text{id} \{ \text{print}(\text{id.name}) \} + \text{id} \{ \text{print}(\text{id.name}) \} \{ \text{print}("+") \} + T \{ \text{print}("+") \} R$
- $\rightarrow \text{id} \{ \text{print}(\text{id.name}) \} + \text{id} \{ \text{print}(\text{id.name}) \} \{ \text{print}("+") \} + \text{id} \{ \text{print}(\text{id.name}) \} \{ \text{print}("+") \} R$
- $\rightarrow \text{id} \{ \text{print}(\text{id.name}) \} + \text{id} \{ \text{print}(\text{id.name}) \} \{ \text{print}("+") \} + \text{id} \{ \text{print}(\text{id.name}) \} \{ \text{print}("+") \} \varepsilon$

$E \rightarrow T R$

$R \rightarrow + T \{ \text{print}("+") \} R1$

$R \rightarrow \varepsilon$

$T \rightarrow \text{id} \{ \text{print}(\text{id.name}) \}$

How right-most derivation?
Do it by yourself.

Top-Down Translation



- We will look at the implementation of L-attributed definitions during predictive parsing.
- Instead of the syntax-directed translations, we will work with translation schemes.
- We will see how to evaluate inherited attributes (in L-attributed definitions) during recursive predictive parsing.
- We will also look at what happens to attributes during the left-recursion elimination in the left-recursive grammars.

Top-Down Translation (cont.)

◆ For each nonterminal,

- ◆ inherited attributes $\rightarrow$ formal parameters
- ◆ synthesized attributes $\rightarrow$ returned values

◆ For each production,

- ◆ for each terminal X with synthesized attribute x ,
save $X.x$; match(X);
- ◆ for nonterminal B , $c := B(b_1, b_2, \dots, b_k)$;
- ◆ for each semantic rule, copy the rule to the parser

A Translation Scheme with Inherited Attributes

$$D \rightarrow T \text{ id} \{ \text{addtype}(\text{id.entry}, T.\text{type}), L.\text{in} = T.\text{type} \} L$$
$$T \rightarrow \text{int} \{ T.\text{type} = \text{integer} \}$$
$$T \rightarrow \text{real} \{ T.\text{type} = \text{real} \}$$
$$L \rightarrow \text{id} \{ \text{addtype}(\text{id.entry}, L.\text{in}), L_1.\text{in} = L.\text{in} \} L_1$$
$$L \rightarrow \varepsilon$$

- This is a translation scheme for an L-attributed definitions.

Predictive Parsing (of Inherited Attributes)

```
procedure D() {  
    int Ttype, Lin, identry;  
    call T(&Ttype); consume(id, &identry);  
    addtype(identry, Ttype); Lin=Ttype;  
    call L(Lin);  
}  
procedure T(int *Ttype) {  
    if (currtoken is int) { consume(int); *Ttype=TYPEINT; }  
    else if (currtoken is real) { consume(real); *Ttype=TYPEREAL; }  
    else { error("unexpected type"); }  
}  
procedure L(int Lin) {  
    if (currtoken is id) { int Llin, identry; consume(id, &identry);  
        addtype(identry, Lin); Llin=Lin; call L(Llin); }  
    else if (currtoken is endmarker) { }  
    else { error("unexpected token"); }  
}
```

a synthesized attribute (an output parameter)

an inherited attribute (an input parameter)



Eliminating Left Recursion from Translation Scheme

- A translation scheme with a left recursive grammar.

$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val \}$
$E \rightarrow E_1 - T$	$\{ E.val = E_1.val - T.val \}$
$E \rightarrow T$	$\{ E.val = T.val \}$
$T \rightarrow T_1 * F$	$\{ T.val = T_1.val * F.val \}$
$T \rightarrow F$	$\{ T.val = F.val \}$
$F \rightarrow (E)$	$\{ F.val = E.val \}$
$F \rightarrow \text{digit}$	$\{ F.val = \text{digit.lexval} \}$

- When we eliminate the left recursion from the grammar (to get a suitable grammar for the top-down parsing) we also have to change semantic actions

Eliminating Left Recursion (cont.)



inherited attribute

synthesized attribute

$E \rightarrow T \{ E'.in = T.val \} E' \{ E.val = E'.syn \}$

$E' \rightarrow + T \{ E'_1.in = E'.in + T.val \} E'_1 \{ E'.syn = E'_1.syn \}$

$E' \rightarrow - T \{ E'_1.in = E'.in - T.val \} E'_1 \{ E'.syn = E'_1.syn \}$

$E' \rightarrow \varepsilon \{ E'.syn = E'.in \}$

$T \rightarrow F \{ T'.in = F.val \} T' \{ T.val = T'.syn \}$

$T' \rightarrow * F \{ T'_1.in = T'.in * F.val \} T'_1 \{ T'.syn = T'_1.syn \}$

$T' \rightarrow \varepsilon \{ T'.syn = T'.in \}$

$F \rightarrow (E) \{ F.val = E.val \}$

$F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

$E \rightarrow E1 + T \{ E.val = E1.val + T.val \}$

$E \rightarrow E1 - T \{ E.val = E1.val - T.val \}$

$E \rightarrow T \{ E.val = T.val \}$

$T \rightarrow T1 * F \{ T.val = T1.val * F.val \}$

$T \rightarrow F \{ T.val = F.val \}$

$F \rightarrow (E) \{ F.val = E.val \}$

$F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

Left-most dereivation with semantic rules Cf. p.303 fig.5.26 expression 9-5

- $E \rightarrow T \{ E'.in = T.val \} E' \{ E.val = E'.syn \}$
- $\rightarrow F \{ T'.in = F.val \} T' \{ T.val = T'.syn \} \{ E'.in = T.val \} E' \{ E.val = E'.syn \}$
- $\rightarrow \text{digit} \{ F.val = \text{digit.lexval} \} \{ T'.in = F.val \} T' \{ T.val = T'.syn \} \{ E'.in = T.val \} E' \{ E.val = E'.syn \}$
- $\rightarrow \text{digit} \{ F.val = \text{digit.lexval} \} \{ T'.in = F.val \} \epsilon \{ T'.syn = T'.in \} \{ T.val = T'.syn \} \{ E'.in = T.val \} E' \{ E.val = E'.syn \}$
- $\rightarrow \text{digit} \{ F.val = \text{digit.lexval} \} \{ T'.in = F.val \} \epsilon \{ T'.syn = T'.in \} \{ T.val = T'.syn \} \{ E'.in = T.val \} - T \{ E'1.in = E'.in - T.val \} E'1 \{ E'.syn = E'1.syn \} \{ E.val = E'.syn \}$
- $\rightarrow \text{digit} \{ F.val = \text{digit.lexval} \} \{ T'.in = F.val \} \epsilon \{ T'.syn = T'.in \} \{ T.val = T'.syn \} \{ E'.in = T.val \} - F \{ T'.in = F.val \} T' \{ T.val = T'.syn \} \{ E'1.in = E'.in - T.val \} E'1 \{ E'.syn = E'1.syn \} \{ E.val = E'.syn \}$
- $\rightarrow \text{digit} \{ F.val = \text{digit.lexval} \} \{ T'.in = F.val \} \epsilon \{ T'.syn = T'.in \} \{ T.val = T'.syn \} \{ E'.in = T.val \} - \text{digit} \{ F.val = \text{digit.lexval} \} \{ T'.in = F.val \} T' \{ T.val = T'.syn \} \{ E'1.in = E'.in - T.val \} E'1 \{ E'.syn = E'1.syn \} \{ E.val = E'.syn \}$
- $\rightarrow \text{digit} \{ F.val = \text{digit.lexval} \} \{ T'.in = F.val \} \epsilon \{ T'.syn = T'.in \} \{ T.val = T'.syn \} \{ E'.in = T.val \} - \text{digit} \{ F.val = \text{digit.lexval} \} \{ T'.in = F.val \} \epsilon \{ T'.syn = T'.in \} \{ T.val = T'.syn \} \{ E'1.in = E'.in - T.val \} E'1 \{ E'.syn = E'1.syn \} \{ E.val = E'.syn \}$
- $\rightarrow \text{digit} \{ F.val = \text{digit.lexval} \} \{ T'.in = F.val \} \epsilon \{ T'.syn = T'.in \} \{ T.val = T'.syn \} \{ E'.in = T.val \} - \text{digit} \{ F.val = \text{digit.lexval} \} \{ T'.in = F.val \} \epsilon \{ T'.syn = T'.in \} \{ T.val = T'.syn \} \{ E'1.in = E'.in - T.val \} \epsilon \{ E'.syn = E'.in \} \{ E'.syn = E'1.syn \} \{ E.val = E'.syn \}$

Eliminating Left Recursion (in general)



$A \rightarrow A_1 Y \{ A.a = g(A_1.a, Y.y) \}$ a left recursive grammar with
 $A \rightarrow X \{ A.a = f(X.x) \}$ synthesized attributes (a, y, x) .

$\Downarrow$ eliminate left recursion

inherited attribute of the new non-terminal

synthesized attribute of the new non-

terminal

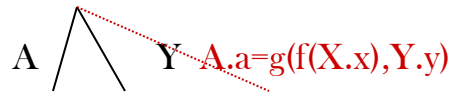
$A \rightarrow X \{ A'.in = f(X.x) \} A' \{ A.a = A'.syn \}$
 $A' \rightarrow Y \{ A'_1.in = g(A'.in, Y.y) \} A'_1 \{ A'.syn = A'_1.syn \}$
 $A' \rightarrow \varepsilon \{ A'.syn = A'.in \}$

Evaluating attributes



A

parse tree of left recursive grammar

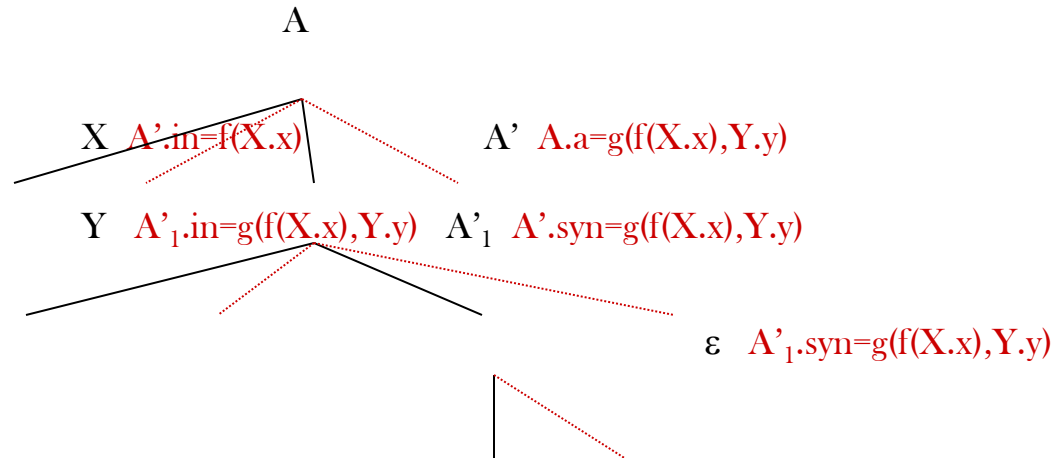


grammar

X

$X.x = f(X.x)$

parse tree of non-left-recursive



Bottom-Up Translation

- ◆ We discussed already “top-down translation.”
- ◆ Works exceptionally well with L-attributed Definitions (with corresponding translation schemes)
- ◆ Nevertheless we must make sure that the underlying grammar is suitable for predictive parsing.
- ◆ Grammar has no left-recursion and it is left-factored.
- ◆ We also discussed “bottom up translation” for S-Directed Definitions.
- ◆ Simple Idea: Use additional stack space to store attribute values for Non-terminals. During Reduce Actions update attributes in stack accordingly.

Removing Embedding Semantic Actions



- In bottom-up evaluation scheme, the semantic actions are evaluated during the reductions.
- During the bottom-up evaluation of S-attributed definitions, we have a parallel stack to hold synthesized attributes.
- *Problem:* where are we going to hold inherited attributes?
- *A Solution:*
 - We will convert our grammar to an equivalent grammar to guarantee to the followings.
 - All embedding semantic actions in our translation scheme will be moved into the end of the production rules.
 - All inherited attributes will be copied into the synthesized attributes (most of the time synthesized attributes of new non-terminals).
 - Thus we will evaluate all semantic actions during reductions, and we find a place to store an inherited attribute.

Removing Embedding Semantic Actions

- To transform our translation scheme into an equivalent translation scheme:
 1. Remove an embedding semantic action S_i , put new a non-terminal M_i instead of that semantic action.
 2. Put that semantic action S_i into the end of a new production rule $M_i \rightarrow \varepsilon$ for that non-terminal M_i .
 3. That semantic action S_i will be evaluated when this new production rule is reduced.
 4. The evaluation order of the semantic rules are not changed by this transformation.

Removing Embedding Semantic Actions

$$A \rightarrow \{S_1\} X_1 \{S_2\} X_2 \dots \{S_n\} X_n$$

$\Downarrow$ remove embedding semantic actions

$$A \rightarrow \mathbf{M}_1 X_1 \mathbf{M}_2 X_2 \dots \mathbf{M}_n X_n$$

$$M_1 \rightarrow \varepsilon \{S_1\}$$

$$M_2 \rightarrow \varepsilon \{S_2\}$$

.

.

$$M_n \rightarrow \varepsilon \{S_n\}$$

Removing Embedding Semantic Actions

$$E \rightarrow T R$$
$$R \rightarrow + T \{ \text{print}(\text{"+"}) \} R_1$$
$$R \rightarrow \varepsilon$$
$$T \rightarrow \text{id} \{ \text{print}(\text{id.name}) \}$$

$\Downarrow$ remove embedding semantic actions

$$E \rightarrow T R$$
$$R \rightarrow + T M R_1$$
$$R \rightarrow \varepsilon$$
$$T \rightarrow \text{id} \{ \text{print}(\text{id.name}) \}$$
$$M \rightarrow \varepsilon \{ \text{print}(\text{"+"}) \}$$



Evaluation of Inherited Attributes

Removing embedding actions from attribute grammar by introducing **marker** nonterminals

$$\begin{aligned} E &\rightarrow T R \\ R &\rightarrow "+" T \{ \text{print}('+') \} R \mid "-" T \{ \text{print}('-') \} R \mid \varepsilon \\ T &\rightarrow \text{num} \{ \text{print}(\text{num.val}) \} \end{aligned}$$
$$\begin{aligned} E &\rightarrow T R \\ R &\rightarrow "+" T \text{ M } R \mid "-" T \text{ N } R \mid \varepsilon \\ T &\rightarrow \text{num} \{ \text{print}(\text{num.val}) \} \\ \text{M} &\rightarrow \varepsilon \{ \text{print}('+') \} \\ \text{N} &\rightarrow \varepsilon \{ \text{print}('-') \} \end{aligned}$$

Evaluation of Inherited Attributes (cont.)

- Inheriting synthesized attributes on the stack

$$A \rightarrow X \{Y.i := X.s\} Y$$

symbol	val
...	...
X	X.s
Y	

top →

$$A \rightarrow X MY$$

$$M \rightarrow \varepsilon \{Y.i := X.s\}$$

- If inherited attribute $Y.i$ is defined by the copy rule $Y.i = X.s$, then the value $X.s$ can be used where $Y.i$ is called for.
- Copy rules play an important role in the evaluation of inherited attributes.

An Example (P.311)


$$\begin{aligned} D &\rightarrow T \{L.in := T.type;\} L \\ T &\rightarrow \text{int} \quad \{T.type := \text{integer};\} \\ T &\rightarrow \text{float} \quad \{T.type := \text{float};\} \\ L &\rightarrow \{L_1.in := L.in\} \quad L_1, \text{id} \quad \{\text{addtype}(\text{id.entry}, L.in);\} \\ L &\rightarrow \text{id} \quad \{\text{addtype}(\text{id.entry}, L.in);\} \end{aligned}$$

$$\begin{aligned} D &\rightarrow T \ L \\ T &\rightarrow \text{int} \quad \{\text{val}[\text{ntop}] := \text{integer};\} \\ T &\rightarrow \text{float} \quad \{\text{val}[\text{ntop}] := \text{float};\} \\ L &\rightarrow L_1, \text{id} \quad \{\text{addtype}(\text{val}[\text{top}], \text{val}[\text{top}-3]);\} \\ L &\rightarrow \text{id} \quad \{\text{addtype}(\text{val}[\text{top}], \text{val}[\text{top}-1]);\} \end{aligned}$$

An Example



Input	symbol	val	production used
int p,q,r			
p,q,r	int		
p,q,r	T	$\bar{i}$	$T \rightarrow \text{int}$
,q,r	T id	$i\ e$	
,q,r	T L	$i_$	$L \rightarrow \text{id}$
q,r	T L ,	$i_ _$	
,r	T L , id	$i_ _ e$	
,r	T L	$i_$	$L \rightarrow L , \text{id}$
r	T L ,	$i_ _$	
	T L , id	$i_ _ e$	
	T L	$i_$	$L \rightarrow L , \text{id}$
	D		

Inheriting the value of a synthesized attribute works only if the grammar allows the position of the attribute value to be predicted.

Another Example


$$\begin{aligned} S &\rightarrow a A C \quad \{C.i := A.s\} \\ S &\rightarrow b A B C \quad \{C.i := A.s\} \\ C &\rightarrow c \quad \{C.s := g(C.i)\} \end{aligned}$$
$$\begin{aligned} S &\rightarrow a A C \quad \{C.i := A.s\} \\ S &\rightarrow b A B M C \quad \{M.i := A.s; C.i := M.s\} \\ C &\rightarrow c \quad \{C.s := g(C.i)\} \\ M &\rightarrow \varepsilon \quad \{M.s := M.i\} \end{aligned}$$

Another Example

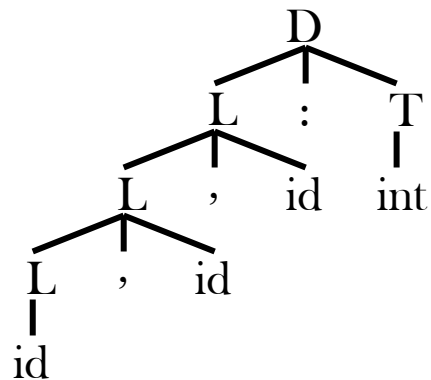


$$S \rightarrow a A C \quad \{C.i := f(A.s)\}$$

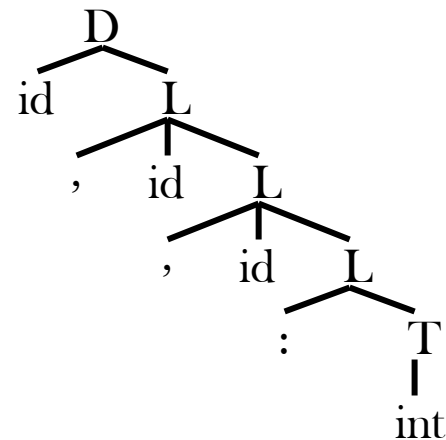
$$\begin{aligned} S &\rightarrow a A N C \quad \{N.i := A.s; C.i := N.s\} \\ N &\rightarrow \varepsilon \quad \{N.s := f(N.i)\} \end{aligned}$$

From Inherited to Synthesized

$D \rightarrow L : T$
 $L \rightarrow L , id \mid id$
 $T \rightarrow integer \mid char$



$D \rightarrow id L$
 $L \rightarrow , id L \mid : T$
 $T \rightarrow integer \mid char$



Problems



- Some L-attributed definitions based on LR grammars cannot be evaluated during bottom-up parsing.

$S \rightarrow \{ L.i=0 \} L$

→ this translations scheme cannot be implemented

$L \rightarrow \{ L_1.i=L.i+1 \} L_1 \ 1$

during the bottom-up parsing

$L \rightarrow \varepsilon \{ \text{print}(L.i) \}$

$S \rightarrow M_1 L$

$L \rightarrow M_2 L_1 \ 1$

→ But since $L \rightarrow \varepsilon$ will be reduced first by the bottom-up

$L \rightarrow \varepsilon \quad \{ \text{print}(s[\text{top}]) \}$ parser, the translator cannot know the number of 1s.

$M_1 \rightarrow \varepsilon \quad \{ s[\text{ntop}]=0 \}$

$M_2 \rightarrow \varepsilon \quad \{ s[\text{ntop}]=s[\text{top}]+1 \}$

Problems



- The modified grammar cannot be LR grammar anymore.

$L \rightarrow L b$

$L \rightarrow M L b$

$L \rightarrow a$



$L \rightarrow a$

NOT LR-grammar

$M \rightarrow \varepsilon$

$S' \rightarrow \bullet L, \$$

$L \rightarrow \bullet M L b, \$$

$L \rightarrow \bullet a, \$$

$M \rightarrow \bullet, a \rightarrow \text{shift/reduce conflict}$



1、写出下面文法的属性文法：

$$\text{Number} \rightarrow \text{Number}_1 \text{ Digit}$$
$$\text{Number} \rightarrow \text{Digit}$$
$$\text{Digit} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \mid$$

2、写出下面文法的属性文法：

$$\text{Number} \rightarrow \text{Digit Number}_1$$
$$\text{Number} \rightarrow \text{Digit}$$
$$\text{Digit} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \mid$$



3、十进制浮点数的文法修改如下：

$$\text{dnum} \rightarrow \text{num}.\text{snum}$$
$$\text{num} \rightarrow \text{num}_1 \text{ digit} \mid \text{digit}$$
$$\text{snum} \rightarrow \text{digit snum}_1 \mid \text{digit}$$
$$\text{digit} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

(注意：整数、小数分别用左、右递归文法)